

C1

A 小水獭和主定理

小水獭正在学习分治和递归，在计算复杂度时用到了主定理，希望你可以帮助它写一个程序来计算满足特定形式递归式的算法复杂度。

具体来说，给定正整数 a, b, k ，所求递归式为：

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^k)$$

根据主定理，有：

$$T(n) = \begin{cases} O(n^{\log_b a}), & \log_b a > k \\ O(n^k \log n), & \log_b a = k \\ O(n^k), & \log_b a < k \end{cases}$$

输入格式

第一行一个正整数 t ($1 \leq t \leq 2 \times 10^5$)，表示数据组数。

对于每组数据，一行三个正整数 a, b, k ($1 \leq a, k \leq 10^9, 2 \leq b \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行：

- 若 $T(n) = O(n^{\log_b a})$ ，输出 $n^{\{\log_ba\}}$ 。
- 若 $T(n) = O(n^k \log n)$ ，输出 $n^k \log n$ 。
- 若 $T(n) = O(n^k)$ ，输出 n^k 。

```
#include <bits/stdc++.h>
using namespace std;
int t, a, b, k;
int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d%d", &a, &b, &k);
        long long x = 1;
        if(b > 1){
            for(int i = 1; i <= k && x <= a; i++) x *= b;
        }
        if(a == x) puts("n^k\\log n");
        else if(a > x) puts("n^{\log\_ba}");
        else puts("n^k");
    }
}
```

B 一元多项式计算器

Zhoues 对《算法导论》这本书简直是爱不释手，当他翻到这本书的第 23 页准备暗中观察时，立马被 Horner 规则给吸引到了，于是他打算做一个基于 Horner 规则的一元多项式计算器，帮助他进行一类特殊的二元多项式。

具体来说，给定如下两个一元多项式：

$$\sum_{i=0}^n a_i x^i = a_0 + a_1 x + a_2 x^2 + \dots + a_n x^n$$
$$\sum_{i=0}^m b_i y^i = b_0 + b_1 y + b_2 y^2 + \dots + b_m y^m$$

定义如下的二元多项式：

$$f(x,y) = \left(\sum_{i=0}^n a_i x^i \right) \left(\sum_{i=0}^m b_i y^i \right)$$

你需要处理 q 次计算，第 i 次计算给定两个变量值 X_i 和 Y_i ，你需要求解 $f(X_i, Y_i) \bmod 10007$ 。

输入格式

第一行一个正整数 n ($1 \leq n \leq 3 \times 10^4$)，表示第一个一元多项式的次数。

第二行 $n+1$ 个非负整数 $a_0, a_1, \dots, a_n$ ($0 \leq a_i \leq 10^3$)，表示第一个一元多项式的系数。

第三行一个正整数 m ($1 \leq m \leq 3 \times 10^4$)，表示第二个一元多项式的次数。

第四行 $m+1$ 个非负整数 $b_0, b_1, \dots, b_m$ ($0 \leq b_i \leq 10^3$)，表示第二个一元多项式的系数。

第五行一个正整数 q ($1 \leq q \leq 2.5 \times 10^3$)，表示计算的次数。

接下来 q 行，第 i 行两个非负整数 X_i, Y_i ($0 \leq X_i, Y_i \leq 10^4$)，表示第 i 次计算给定的两个变量值。

输出格式

输出 q 行，第 i 行一个非负整数，表示 $f(X_i, Y_i) \bmod 10007$ 。

```
#include<stdio.h>
#define M 10007
long long a[1000001],b[1000001];
long long n,m,x,y,i,t,c;
long long Horner(long long *a,long long n,long long x) {
    if(n<1)
        return a[0] % M;
    else
        return (a[0] % M + (Horner(a+1,n-1,x) * x) % M) % M;
}

int main() {
    scanf("%lld",&n);
    for(i=0; i<=n; i++) {
        scanf("%lld",&a[i]);
    }
    scanf("%lld",&m);
    for(i=0; i<=m; i++) {
        scanf("%lld",&b[i]);
    }
    scanf("%lld",&c);
    while(c--)
    {
        scanf("%lld %lld",&x,&y);
        printf("%lld\n",( Horner(a,n,x) % M) * (Horner(b,m,y) % M) ) % M );
    }
    return 0;
}
```

C 小水獭和模运算

小水獭正在学习模运算，它通过 Hint 发现 $ab \bmod c = (a \bmod c)(b \bmod c) \bmod c$ 。

现在小水獭有 n 个非负整数 $a_1, a_2, \dots, a_n$ ，它定义 $f(i)$ 为除了 a_i 以外的 $n - 1$ 个非负整数的乘积对 $10^9 + 6$ 取模后的结果，形式化地说：

$$f(i) = \left(\prod_{j=1, j \neq i}^n a_j \right) \bmod (10^9 + 6)$$

小水獭希望你可以帮助它写一个程序计算 $f(1), f(2), \dots, f(n)$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($2 \leq n \leq 10^5$)，含义同题目描述。

第二个 n 个非负整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 10^9 + 6$)，含义同题目描述。

输出格式

对于每组数据，输出一行 n 个非负整数，表示 $f(1), f(2), \dots, f(n)$ 。

```
#include <bits/stdc++.h>
#define maxn 100086
using namespace std;
const int p = 1e9 + 6;
int t, n;
int a[maxn], f[maxn], g[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
        f[0] = 1; for(int i = 1; i <= n; i++) f[i] = 1ll * f[i - 1] * a[i] % p;
        g[n + 1] = 1; for(int i = n; i >= 1; i--) g[i] = 1ll * g[i + 1] * a[i] % p;
        for(int i = 1; i <= n; i++) printf("%d ", 1ll * f[i - 1] * g[i + 1] % p); puts("");
    }
}
```

D 日期统计

本题中所使用的纪年体系为公元，仅考虑年份位于 [2000, 9999] 的日期。

所有日期均可以表示为一个唯一的 8 位十进制数字，前四位表示年，中间两位表示月，最后两位表示日，例如 2022 年 9 月 6 日可以表示为 20220906。

一个日期是好的当且仅当这个日期按上述规则表示为 8 位十进制数字 $a_1a_2a_3a_4a_5a_6a_7a_8$ 时满足下面两个条件中至少一条：

- $\exists i \in \{1, 2, \dots, 8\}$ 使得 $\forall 1 \leq j < i, a_j \leq a_{j+1}$ 并且 $\forall i < j \leq n, a_j \leq a_{j-1}$ 。
- $\exists i \in \{1, 2, \dots, 8\}$ 使得 $\forall 1 \leq j < i, a_j \geq a_{j+1}$ 并且 $\forall i < j \leq n, a_j \geq a_{j-1}$ 。

比如 20000123 就是一个好的日期。小水獭想知道对于给定两个日期，这两个日期之间（包含这两个日期）共有多少个好日期。

输入格式

第一行一个 8 位十进制数字，表示第一个日期。

第二行一个 8 位十进制数字，表示第二个日期。

保证第一个日期在时间上位于第二个日期之前，并且两个日期均是年份位于 [2000, 9999] 的合法日期。

输出格式

一行一个非负整数，表示两个日期间好日期的个数。

```

#include <bits/stdc++.h>
#define inf 0x3f3f3f3f
#define eps 1e-14
#define maxn 300005
typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
using namespace std;
inline int read(){
    int x = 0, f = 1; char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1; ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0'; ch = getchar();}
    return x * f;
}

char s[10], t[10];
int y[2], m[2], d[2];
int mx[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

bool check(int a, int b, int c){
    int s[10], t = 0;
    for(int i = 1; i <= 2; i++) s[++t] = c % 10, c /= 10;
    for(int i = 1; i <= 2; i++) s[++t] = b % 10, b /= 10;
    for(int i = 1; i <= 4; i++) s[++t] = a % 10, a /= 10;
    for(int i = 1; i <= 8; i++){
        bool f = 1;
        for(int j = i + 1; j <= 8; j++) if(s[j] < s[j - 1]) f = 0;
        for(int j = i - 1; j >= 1; j--) if(s[j] < s[j + 1]) f = 0;
        if(f) return 1;
        f = 1;
        for(int j = i + 1; j <= 8; j++) if(s[j] > s[j - 1]) f = 0;
        for(int j = i - 1; j >= 1; j--) if(s[j] > s[j + 1]) f = 0;
        if(f) return 1;
    }
    return 0;
}

bool isleap(int x){
    return (x % 4 == 0 && x % 100 != 0) || (x % 400 == 0);
}

void solve(){
    scanf("%s", s + 1), scanf("%s", t + 1);
    for(int i = 1; i <= 4; i++) y[0] = y[0] * 10 + s[i] - '0';
    for(int i = 5; i <= 6; i++) m[0] = m[0] * 10 + s[i] - '0';
    for(int i = 7; i <= 8; i++) d[0] = d[0] * 10 + s[i] - '0';
    for(int i = 1; i <= 4; i++) y[1] = y[1] * 10 + t[i] - '0';
    for(int i = 5; i <= 6; i++) m[1] = m[1] * 10 + t[i] - '0';
    for(int i = 7; i <= 8; i++) d[1] = d[1] * 10 + t[i] - '0';
    int ans = 0;
    while(y[0] != y[1] || m[0] != m[1] || d[0] != d[1]){
        ans += check(y[0], m[0], d[0]);
        d[0]++;
        if(d[0] > mx[m[0]] + (m[0] == 2 && isleap(y[0]))) m[0]++, d[0] = 1;
        if(m[0] > 12) m[0] = 1, y[0]++;
    }
    ans += check(y[1], m[1], d[1]);
}

```

```

        printf("%d\n", ans);
    }

    int main(){
        int t = 1, cnt = 1;
        while(t--){
            solve();
        }

        return 0;
    }

```

E 单词游戏-青春版

四六级考试将至，the_ignorant、Zhoues、2sozx 为提高英语水平，便想了一个单词游戏。这个游戏一共有 t 轮，且每轮游戏相互独立。

每轮游戏中，三人每人说出 n 个单词。最终，对于每个被说出的英文单词：

- 若其恰好被一人说出，则此人加 3 分。
- 若其恰好被两人说出，则两人各加 1 分。
- 若其被三人均说出，则三人均不加分。

the_ignorant 想知道他们每轮最后得了多少分。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示游戏轮数。

对于每轮游戏，第一行一个正整数 n ($1 \leq n \leq 10^5$)，表示单词数量。

接下来三行，每行 n 个互不相同的非空小写字母单词，且长度不超过 4，表示每个人此轮说出的单词。

输出格式

对于每轮游戏，输出一行三个非负整数，代表三个人的最后得分。

```

#include<bits/stdc++.h>
using namespace std;
int a[600000][3];
char s[10];
int ans[3];
int tr(char s[])
{
    int res=0;
    int len=strlen(s);
    for(int i=0;i<len;i++)
    {
        res=res*27+s[i]-'a'+1;
    }
    return res;
}
int main()
{
    int t,n;
    scanf("%d",&t);
    while(t--)
    {
        memset(a,0,sizeof(a));
        memset(ans,0,sizeof(ans));
        scanf("%d",&n);
    }
}

```

```

for(int j=0;j<3;j++)
{
    for(int i=0;i<n;i++)
    {
        scanf("%s",s);
        a[tr(s)][j]++;
    }
}
for(int i=0;i<27*27*27*27;i++)
{
    if(a[i][0]&&a[i][1]&&a[i][2])
    {
        continue;
    }
    else if(a[i][0]&&a[i][1])
    {
        ans[0]++;
        ans[1]++;
    }
    else if(a[i][0]&&a[i][2])
    {
        ans[0]++;
        ans[2]++;
    }
    else if(a[i][1]&&a[i][2])
    {
        ans[1]++;
        ans[2]++;
    }
    else if(a[i][0])
    {
        ans[0]+=3;
    }
    else if(a[i][1])
    {
        ans[1]+=3;
    }
    else if(a[i][2])
    {
        ans[2]+=3;
    }
}
for(int i=0;i<3;i++)
{
    printf("%d ",ans[i]);
}
puts("");
}
}

```

F 逆序 k 倍对

Zhoues 很喜欢研读《算法导论》，有一天在暗中观察该书第 24 页的时候，想到了一个问题并需要你来帮他解决，问题如下：

- 给定一个序列 $a_1, a_2, \dots, a_n$ 和一个正整数 k ，如果 $1 \leq i < j \leq n$ 且 $a_i > k \cdot a_j$ 我们就将 (i, j) 称作一个**逆序 k 倍对**。请你计算序列中逆序 k 倍对的个数。

输入格式

第一行两个正整数 n, k ($1 \leq n \leq 10^5, 1 \leq k \leq 10$)，含义同题目描述。

第二个 n 个正整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i < 2^{31}$)，含义同题目描述。

对于得分占比 10% 的测试点，保证 $1 \leq n \leq 100$ 。

输出格式

一行一个非负整数，表示序列中逆序 k 倍对的个数。

```
#include<stdio.h>
int a[1000002], n, k, num;

long long reversePairsRecursive(int* nums, int left, int right) {
    if (left == right) {
        return 0;
    } else {
        int mid = (left + right) / 2;
        int n1 = reversePairsRecursive(nums, left, mid);
        int n2 = reversePairsRecursive(nums, mid + 1, right);
        long long ret = n1 + n2;

        int i = left;
        int j = mid + 1;
        while (i <= mid) {
            while (j <= right && (long long)nums[i] > k * (long long)nums[j])
                j++;
            ret += (j - mid - 1);
            i++;
        }

        int sorted[right - left + 1];
        int p1 = left, p2 = mid + 1;
        int p = 0;
        while (p1 <= mid || p2 <= right) {
            if (p1 > mid) {
                sorted[p++] = nums[p1++];
            } else if (p2 > right) {
                sorted[p++] = nums[p2++];
            } else {
                if (nums[p1] < nums[p2]) {
                    sorted[p++] = nums[p1++];
                } else {
                    sorted[p++] = nums[p2++];
                }
            }
        }
        for (int i = 0; i < right - left + 1; i++) {
            nums[left + i] = sorted[i];
        }
        return ret;
    }
}

int reversePairs(int* nums, int numsSize) {
```

```

    if (numssize == 0) {
        return 0;
    }
    return reversePairsRecursive(nums, 0, numssize - 1);
}

int main() {
    scanf("%d %d",&n,&k);
    for(int i = 0; i < n ; i++) {
        scanf("%d",&a[i]);
    }
    printf("%lld\n",reversePairs(a,n));
    return 0;
}

```

G 暗中观察

Zhoues 这个同学估计大家都不怎么认识，但是这不重要，重要的是你只需要知道他有一项特殊的能力——**暗中观察**。他经常通过这个能力轻松看透一些事物背后的算法原理。

虽然学校因为疫情封校，但是却校园内却盛行一款游戏：

- 该游戏中有小 A 和小 B 两个角色，初始给定两个正整数 n 和 k ，小 A 和小 B 轮流对 n 减去 k 的任意非负整数次幂（即 $k^i, i \in \mathbb{N}$ ），但需要保证 n 始终非负，最终在自己回合将 n 变为 0 的一方获胜。

每次游戏开始都是小 A 先手，且小 A 小 B 两个人都会采取最优策略，那么每轮谁会赢呢？

Zhoues 暗中观察了几场这两款游戏的对局，已经看透了背后的算法奥秘，现在你只要告诉他游戏的初始情况，他就可以直接告诉你最后的赢家是谁！

输入格式

第一行一个正整数 t ($1 \leq t \leq 50$)，表示数据组数。

对于每组数据，一行两个正整数 n, k ($1 \leq n, k \leq 10^3$)，含义同题目描述。

输出格式

对于每组数据，输出一行：

- 若小 A 获胜，输出 `~A~`。
- 若小 B 获胜，输出 `~B~`。

```

#include <bits/stdc++.h>
#define maxn 1005
using namespace std;

int ans[maxn], vis[maxn];
int n, k;

int play(int x) {
    if(!x) return 0;
    if(vis[x]) return ans[x];
    vis[x] = 1;
    int res = 0;
    for(int i = 1; i <= x; i *= k) {
        res |= !play(x - i);
    }
    ans[x] = res;
    return res;
}

int main() {
    int t;
    scanf("%d",&t);

```

```

while(t--){
    scanf("%d%d",&n,&k);
    if(k == 1 || n < k){
        puts(n%2==0 ? "~B~" : "~A~");
    } else {
        for(int i = 0; i <= n; i++){
            vis[i] = 0, ans[i] = 0;
        }
        puts(play(n) ? "~A~" : "~B~");
    }
}
return 0;
}

```

H XIAO7 想要变成猫猫

XIAO7 每天都在感叹：「我什么时候才能变成猫猫呢，变成猫猫就不用努力了(=> <=)！」

于是天上传来一个声音：「如果你能答对以下若干个问题，你就可以变成猫猫。」

如果 XIAO7 没有如愿以偿地变成猫猫，她就会给大家的 Java 加难度，所以请你帮帮她(=^ ^=)!

具体来说，给定一个长度为 n 的序列 $x_1, x_2, \dots, x_n$ 和若干个问题，每个问题包含四个整数 a, b, c, d ，你需要找到一个最小的整数 $i \in [1, n]$ 满足：

$$a \times (x_i \oplus i)^2 + b \times (x_i \oplus i) \times x_i + c \times x_i^2 + d = 0$$

其中 $\oplus$ 表示按位异或运算，所有数按照 64 位补码进行运算。

为了防止奇怪的离线算法，输入数据进行了加密，假设你读到的四个整数为 a_0, b_0, c_0, d_0 ，则解密方法为

$a = a_0 + lastAns, b = b_0 + lastAns, c = c_0 + lastAns, d = d_0 + lastAns$ ，其中 $lastAns$ 为前一个问题的答案，对于第一个问题 $lastAns = 0$ 。

输入格式

第一行一个整数 n ($1 \leq n \leq 10^5$)，表示序列的长度。

第二行 n 个整数 $x_1, x_2, \dots, x_n$ ($-10^6 \leq x_i \leq 10^6$)，表示给定的序列元素。

接下来若干行，每行四个整数 a_0, b_0, c_0, d_0 ($-10^6 \leq a_0, b_0, c_0 \leq 10^6, -10^{18} \leq d_0 \leq 10^{18}$) 表示一个问题，对于每个问题保证至少存在一个 i 满足条件。

在你回答正确的前提下，当解密出的 a, b, c, d 均为 0 时一定是最后一个问题，此时所有数据都已成功读入。**你不需要对最后一个问题进行输出。**

保证问题的数量不超过 4×10^5 。

输出格式

若干行，第 i 行一个正整数表示第 i 个问题的答案。

注意，解密出 0 0 0 0 的最后一个问题不需要输出。

```

#include<cstdio>
using namespace std;
typedef long long ll;
const int N = 500005;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
ll getll(){
    char c;bool flag=0;ll num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}

```

```

int a[N], b[N], c[N], ans[N];
ll d[N], x[N];
int main(){

    int n = getint(), m = 0;
    for(int i = 1; i <= n; i ++){
        x[i] = getint();

        while(~scanf("%d", &a[++ m]))
            b[m] = getint(), c[m] = getint(), d[m] = getll();
        m --;

        ll j = -a[m --];

        for(int i = m; i; i --){
            ans[i] = j;
            ll A = 1ll * (x[j] ^ j) * (x[j] ^ j);
            ll B = 1ll * (x[j] ^ j) * x[j];
            ll C = 1ll * x[j] * x[j];
            ll t1 = a[i] * A + b[i] * B + c[i] * C + d[i];
            ll t2 = A + B + C + 1;

            j = -t1 / t2;

        }

        for(int i = 1; i <= m; i ++){
            printf("%d\n", ans[i]);
        }
        return 0;
    }
}

```

I 异或游戏

the_ignorant 的单词游戏进行得不是很顺利，于是为了提高数学水平，他又想了一个异或游戏。

他想要找出一个序列，这个序列由 n 个互不相同的非负整数组成，且这 n 个非负整数的按位异或值为 0。

可以证明，在 $n \geq 3$ 时，至少存在一个满足以上条件的序列。但 the_ignorant 不满足于此，他想找出所有满足以上条件的序列中，序列中最大数的最小值。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^4$)，表示数据组数。

对于每组数据，一行一个正整数 n ($3 \leq n \leq 10^6$)，表示序列的长度。

输出格式

对于每组数据，输出一行一个数，代表所有可能序列中，序列中最大数的最小值。

```

#include<bits/stdc++.h>
using namespace std;
int solve(int x)
{
    int res=0;
    while(x>0)
    {
        x>>=1;
        res++;
    }
}

```

```

        return res;
    }
    int main()
    {
        int t,n;
        scanf("%d",&t);
        while(t--)
        {
            scanf("%d",&n);
            if(n%4==0)
            {
                printf("%d\n",n-1);
            }
            else if(n%4==3)
            {
                printf("%d\n",n);
            }
            else if(n%4==1)
            {
                printf("%d\n",n);
            }
            else if(n%4==2)
            {
                if(solve(n+2)>solve(n))
                {
                    printf("%d\n",n+3);
                }
                else
                {
                    printf("%d\n",n+2);
                }
            }
        }
    }
}

```

J 小水獭和二叉树

小水獭正在学习二叉树，它希望世界上所有的树都是二叉树。

具体来说，小水獭有一棵 n 个结点的树 T ，每条边上有一个非负长度。小水獭想要构造一棵 m 个结点的树 T' ，其满足如下限制：

- $n \leq m \leq 2n$ 。
- 每条边上有一个非负长度。
- 任意节点度数不超过 3。
- 对任意节点 $1 \leq i < j \leq n$ ， i 和 j 在 T 和 T' 上的最短路距离相同。

你能帮助小水獭实现它的愿望嘛？

输入格式

第一行一个正整数 t ($1 \leq t \leq 100$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($2 \leq n \leq 100$)，表示树 T 的节点数。

接下来 $n - 1$ 行，每行三个整数 x, y, z ($1 \leq x, y \leq n$, $x \neq y$, $0 \leq z \leq 100$)，表示 T 中存在边 (x, y) 且长度为 z 。

保证给出的边集构成一棵树。

输出格式

对于每组数据，首先输出一行一个正整数 m ($n \leq m \leq 2n$)，表示树 T' 的节点数。

接下来输出 $m - 1$ 行，每行三个整数 x, y, z ($1 \leq x, y \leq m$, $x \neq y$, $0 \leq z \leq 100$)，表示 T' 中存在边 (x, y) 且长度为 z 。

如果有多种可能的答案，你可以输出任意一种。

C2

A 平面最近点对 I

wzf 教给了莫卡很多计算几何的知识，并丢给她一道题作为课后习题。

二维平面上有 n 个点，坐标分别为 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ ，定义两点 (x_i, y_i) 和 (x_j, y_j) 的距离为 $\max(|x_i - x_j|, |y_i - y_j|)$ 。

wzf 要求莫卡求出任意两点间距离的最小值，即：

$$\min_{1 \leq i < j \leq n} \max(|x_i - x_j|, |y_i - y_j|)$$

但是这道题对莫卡来说太难了，于是她想请你帮她解决这道难题。

输入格式

第一行一个正整数 t ($1 \leq t \leq 30$)，表示数据组数。

对于每组数据，一行一个正整数 n ($2 \leq n \leq 10^3$)，表示点的数量。

接下来 n 行，第 i ($1 \leq i \leq n$) 行两个整数 x_i, y_i ($0 \leq |x_i|, |y_i| \leq 10^5$)，表示对应点的坐标。

输出格式

对于每组数据，输出一行一个非负整数，表示任意两点间距离的最小值。

```
#include<bits/stdc++.h>
#define maxn 1005
using ll = long long;
using namespace std;
int t, n, x[maxn], y[maxn];

int main() {
    scanf("%d",&t);
    while(t--) {
        scanf("%d",&n);
```

```

        for(int i = 1; i <= n; i++) scanf("%d", x + i), scanf("%d", y + i);
        int ans = 200000;
        for(int i = 1; i <= n; i++) for(int j = 1; j < i; j++) ans = min(ans,
max(abs(x[i] - x[j]), abs(y[i] - y[j])));
        printf("%d\n", ans);
    }
}

```

B 简单的堆

本题必须使用 C 语言提交。

题目描述

本题是堆的模板题，你的任务是实现一个大根堆，即根节点为所有元素最大值的堆，需要支持插入元素、查询堆顶元素、删除堆顶元素三个操作。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^5$)，表示操作的数量。

接下来 n 行，每行为一个操作，格式如下：

- 1 x ：向堆中插入元素 x ($1 \leq x \leq 10^9$)。
- 2：删除堆顶元素。
- 3：查询堆顶元素。

数据保证后两种操作时堆非空。

输出格式

对于每次查询堆顶元素时，输出一行一个正整数，表示此时堆顶元素的值。

在所有操作结束后，将堆中的元素从大到小依次输出到一行中。

```

#include<stdio.h>
#define maxn 1000005
#define inf 0x3f3f3f3f
#define ls(x) (x << 1)
#define rs(x) (x << 1 | 1)

int n, sz, a[maxn];

void swap(int *a, int *b) {
    int t = *a;
    *a = *b, *b = t;
}

void maxHeapify(int x) {
    int l = ls(x), r = rs(x), larg;
    if(l <= sz && a[l] > a[x]) larg = l;
    else larg = x;
    if(r <= sz && a[r] > a[larg]) larg = r;
    if(larg != x) swap(&a[x], &a[larg]), maxHeapify(larg);
}

void increaseKey(int i, int key) {
    a[i] = key;
    while(i > 1 && a[i / 2] < a[i]) {
        swap(&a[i / 2], &a[i]);
        i = i / 2;
    }
}

```

```

    }
}

void insert(int x) {
    sz += 1;
    a[sz] = -inf;
    increaseKey(sz, x);
}

int top() {
    return a[1];
}

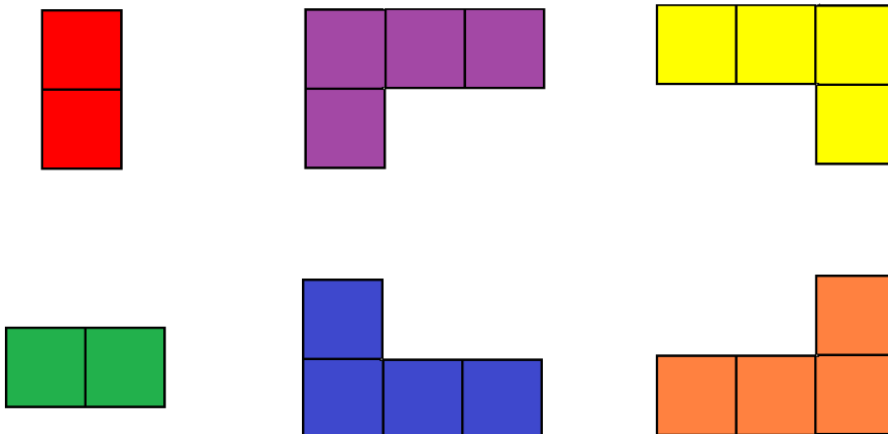
void pop() {
    if(sz < 1) exit(1);
    a[1] = a[sz];
    sz -= 1;
    maxHeapify(1);
}

int main() {
    scanf("%d", &n);
    int i;
    for(i = 1; i <= n; i++) {
        int op, x;
        scanf("%d", &op);
        if(op == 1) {
            scanf("%d", &x);
            insert(x);
        }
        if(op == 2) pop();
        if(op == 3) printf("%d\n", top());
    }
    while(sz) {
        printf("%d ", top()), pop();
    }
}

```

C 简单的铺瓷砖

有一个宽度为 2，长度为 n 的地板需要贴上瓷砖，有以下 6 种形状的瓷砖可供选择：



瓷砖不能**旋转、翻转、重叠、切割**，且必须恰好把地板**铺满**，可以结合输入输出样例理解。

the_ignorant 想知道一共有多少种不同的铺瓷砖方案，两种方案不同当且仅当存在一个位置上的瓷砖颜色不同。由于答案可能很大，你只需要输出答案对 998 244 353 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^6$)，表示数据组数。

对于每组数据，一行一个正整数 n ($1 \leq n \leq 10^6$)，表示地板的长度。

输出格式

对于每组数据，输出一行一个非负整数，表示贴满瓷砖的方案数对 998 244 353 取模后的结果。

```
#include<bits/stdc++.h>
#define maxn 1000005
using ll = long long;
using namespace std;
const int mod = 998244353;

inline ll read() {
    ll x = 0, f = 1;
    char ch = getchar();
    while(ch > '9' || ch < '0') {
        if(ch == '-') f = -1;
        ch = getchar();
    }
    while(ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

int t, tot;
int f[maxn];

int main() {
    f[0] = 1;
    for(int i = 1; i <= 1000000; i++) {
        f[i] = f[i - 1];
        if(i > 1) f[i] = (f[i] + f[i - 2]) % mod;
        if(i > 2) f[i] = (f[i] + 411 * f[i - 3]) % mod;
```

```

        if(i > 3) f[i] = (f[i] + 211 * f[i - 4]) % mod;
    }
    t = read();
    while(t--) printf("%d\n", f[read()]);
}

```

D 小水獭和随机树（简单版）

小水獭正在观摩一棵 n 个节点的树，其中每个点的权值来自一个随机数生成器，树的结构也在代码中直接钦定。它想知道对于每个点来说，所有与其距离小于 k 的祖先中（包括其本身），权值最大点的权值是多少？它的 C++ 代码如下：

```

#include <bits/stdc++.h>
#define maxn 10000086

using namespace std;

unsigned seed;

unsigned Rand(){
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed;
}

int t, n, k;
int fa[maxn];
unsigned a[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d%u", &n, &k, &seed);
        for(int i = 2; i <= n; i++) fa[i] = i - 1;
        for(int i = 1; i <= n; i++) a[i] = Rand();
        unsigned ans = 0;
        for(int i = 1; i <= n; i++){
            unsigned mx = 0;
            int x = i;
            for(int j = 1; j <= k; j++){
                mx = max(mx, a[x]);
                if(x == 1) break;
                x = fa[x];
            }
            ans += mx ^ i;
        }
        printf("%u\n", ans);
    }
}

```

其中 `fa[i]` 表示编号为 `i` 的节点的父节点编号。

可惜这份代码 TLE 了，你能帮它修改一下代码，得到一份在本题数据范围限制下等价但速度更快的代码吗？

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，一行三个整数 $n, k, seed$ ($1 \leq n \leq 10^7$, $1 \leq k \leq n$, $1 \leq seed < 2^{32}$)，表示树的节点数、距离参数和初始随机种子。

输出格式

对于每组数据，按照小水獭原本代码的逻辑，输出一行一个非负整数。

```

#include <bits/stdc++.h>
#define maxn 10000086

using namespace std;

unsigned seed;

unsigned Rand() {
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed;
}

```

```

}

int t, n, k;
int fa[maxn];
unsigned a[maxn];
int q[maxn];

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d%d%u", &n, &k, &seed);
        for(int i = 1; i <= n; i++) fa[i] = i - 1;
        for(int i = 1; i <= n; i++) a[i] = Rand();
        unsigned ans = 0;
        int l = 1, r = 0;
        for(int i = 1; i <= n; i++) {
            while(l <= r && a[i] >= a[q[r]]) r--;
            q[++r] = i;
            if(i - k == q[l]) l++;
            ans += a[q[l]] ^ i;
        }
        printf("%u\n", ans);
    }
}

```

E XIAO7 和兔子

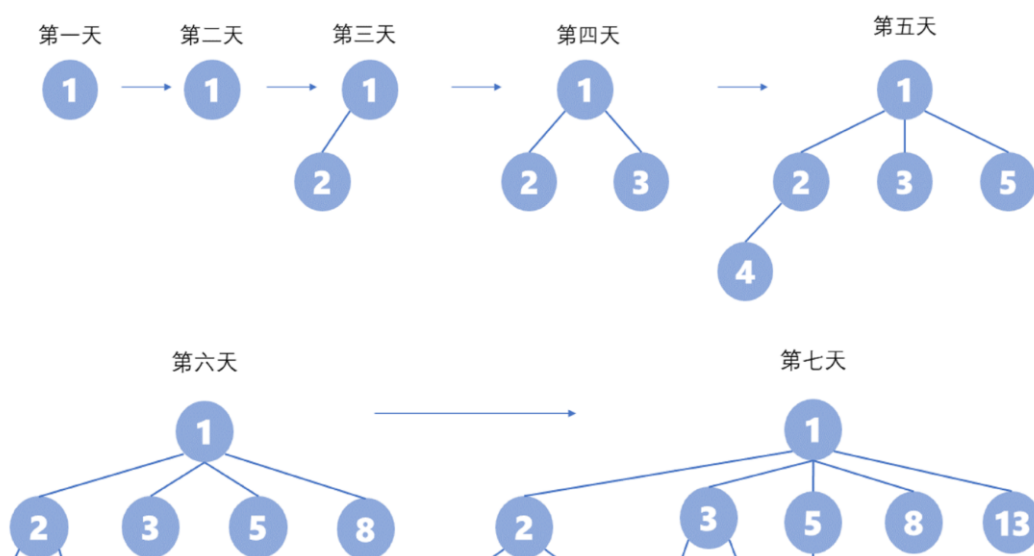
即使上周才下了猫粮雨，XIAO7 的猫粮还是不够吃了，于是她决定在猫娘乐园里养一些兔子吃。

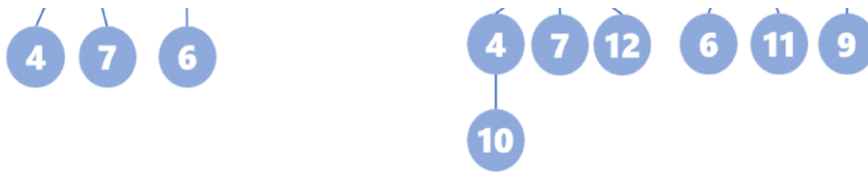
众所周知，兔子的繁殖速率很快，能很好地解决猫粮不够的问题，本题中**假设兔子不会死亡**。

第一天，XIAO7 有一对出生一天的幼年小兔子，小兔子会在**出生后的第二天**起变得成熟，并生育一对新的兔子（即在第 x 天生出的兔子，从 $x + 2$ 天起开始生育）。

XIAO7 懒得给兔子们起名字，因此她决定按兔子的出生顺序给每对兔子编号，假设最初的第一对兔子的编号为 1，如果有两对兔子在同一天出生，那么**父母编号越小的兔子编号越大**。

前 7 天的情况如下图所示：





猫娘「花叶」对此非常难过：「兔兔这么可爱，我不要吃兔兔~」

XIAO7 告诉她「如果你能答对以下若干个问题，我就把兔子们都放了。」

请你帮帮猫娘「花叶」和可爱的兔子们！

具体来说，XIAO7 想知道在 $998^{244}353$ 天后编号为 a 和编号为 b 的兔子的**最近公共祖先**的编号是多少。最近公共祖先是指两对兔子所共有的祖先中，离他们的距离之和最近的一对兔子。

以上图举例来说：

- 编号为 10 和 12 的兔子的最近公共祖先编号为 2。
- 编号为 6 和 13 的兔子的最近公共祖先编号为 1。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^5$)，表示数据组数。

对于每组数据，一行两个正整数 a, b ($1 \leq a, b \leq 2^{61}$)，表示询问的两对兔子的编号。

输出格式

对于每组数据，输出一行一个正整数，表示询问的两对兔子的最近公共祖先编号。

```
#include<iostream>
#include<cstdio>
using namespace std;
typedef long long ll;
const int N = 90;
int getint() {
    char c;
    bool flag=0;
    int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0' && c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}
ll getll() {
    char c;
    bool flag=0;
    ll num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0' && c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}
ll f[N + 5];
ll getfa(ll x) {
    for(int i = N; i; i --)
        if(f[i] < x)
            return f[i - 1] - (x - f[i]) + 1;
    return 1;
}
```

```

}
int main() {
    f[1] = f[2] = 1;
    for(int i = 3; i <= N; i++)
        f[i] = f[i - 1] + f[i - 2];

    int q = getint();
    while(q --) {
        ll x = getll(), y = getll();
        while(x != y)
            x > y ? x = getfa(x) : y = getfa(y);
        cout << x << endl;
    }

    return 0;
}

```

F 小水獭和离散对数

小水獭正在学习离散对数，它正在关注如下问题：

- 给定正整数 $n = 4\,179\,340\,454\,199\,820\,289$ 、 $m = 3$ 和 k ，对于 $i = 1, 2, \dots, 10^5$ ，它想找到最小正整数的 $x \geq k$ 使得 i 在无前导 0 的十进制表示下是 $m^x \bmod n$ 的后缀，或判断这样的 x 不存在。

举例来说，当 $k = 1, i = 11$ 时， $x = 151$ 满足条件，因为 $3^{151} \bmod n = 1\,869\,392\,813\,447\,903\,111$ ，11 是它的后缀，而 $1, 2, \dots, 150$ 均不满足条件。

小水獭数学不太好，希望你能帮它编写一个求解器，你能帮帮它嘛？

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，一行一个正整数 k ($1 \leq k \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行 10^5 个整数，对于第 i ($1 \leq i \leq 10^5$) 个整数：

- 如果对于 i 存在满足条件的 x ，输出 x 。
- 否则，输出 -1 。

```

#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

typedef long long ll;

const ll p = 4179340454199820289ll;

ll fpow(ll x, int y) {
    ll ans = 1;
    while(y) {
        if(y & 1) ans = (__int128)ans * x % p;
        x = (__int128)x * x % p, y >>= 1;
    }
    return ans;
}

int t, k;
int a[maxn];

```

```

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d", &k);
        for(int i = 1; i <= 1e5; i++) a[i] = 0;
        int cnt = 0;
        ll now = fpow(3, k);
        while(cnt < 1e5) {
            ll x = now;
            int y = 0;
            for(int i = 1; i <= 1e5; i *= 10) {
                if(!x) break;
                y += i * (x % 10), x /= 10;
                if(y > 1e5) break;
                if(y && !a[y]) a[y] = k, cnt++;
            }
            now = (__int128)now * 3 % p, k++;
        }
        for(int i = 1; i <= 1e5; i++) printf("%d ", a[i]);
        puts("");
    }
}

```

G 莫卡和计算机网络

wyf 带领莫卡参观了 CVBBBDN 的专属机房，机房的座位可以被视作一个 $n \times n$ 的网格图，一些座位放置了计算机，剩余的座位是空的。

机房的计算机网络结构比较特殊，两个计算机（假设为计算机 i 和计算机 j ，且 $i \neq j$ ）可以通信当且仅当满足下列两个条件中**任意一个**：

- 两台计算机位置相邻，即假设两个计算机的分别位于第 r_i 行第 c_i 列和第 r_j 行第 c_j 列，则有 $|r_i - r_j| + |c_i - c_j| \leq 1$ 。
- 存在 k ($k \neq i, k \neq j$)，满足计算机 i 能够和计算机 k 通信且计算机 j 能够和计算机 k 通信。

参观结束后，莫卡想知道，如果在每一个空位置放上一台计算机，它能够和多少台计算机通信？

输入格式

第一行一个正整数 t ($1 \leq t \leq 20$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 200$)，表示机房座位的行数和列数。

接下来 n 行，第 i ($1 \leq i \leq n$) 行一个长度为 n 的 01 串，如果第 i 个串的第 j ($1 \leq j \leq n$) 位为 1，则代表机房第 i 行第 j 列的座位已经放置了一台计算机，否则代表该座位为空。

输出格式

对于每组数据，第一行输出一个非负整数 m ，代表空位置的个数。

第二行输出 m 个整数，代表每个空位置放上计算机能够通信的计算机数量**从小到大排序**后的结果。

注意，如果空位置的数量为 0，你需要输出一个额外的空行。

```

#include<bits/stdc++.h>
#define maxn 505
using namespace std;

inline int read() {
    int x = 0, f = 1;
    char ch = getchar();
    while(ch > '9' || ch < '0') {
        if(ch == '-') f = -1;
        ch = getchar();
    }
    while(ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
    }
}

```

```

        ch = getchar();
    }
    return x * f;
}

int t, n, mp[maxn][maxn], col[maxn][maxn], cnt[maxn * maxn], now;
char s[maxn];

int dx[4] = {0, 0, 1, -1}, dy[4] = {1, -1, 0, 0};

void dfs(int x, int y) {
    if(col[x][y]) return ;
    col[x][y] = now;
    cnt[now]++;
    for(int i = 0; i < 4; i++) {
        int xx = x + dx[i], yy = y + dy[i];
        if(xx < 1 || xx > n || yy < 1 || yy > n) continue;
        if(mp[xx][yy] == 0) continue;
        if(col[xx][yy]) continue;
        dfs(xx, yy);
    }
}

int main() {
    t = read();
    while(t--) {
        n = read();
        for(int i = 1; i <= n; i++) {
            scanf("%s", s + 1);
            for(int j = 1; j <= n; j++) mp[i][j] = s[j] - '0', cnt[(i - 1) * n +
j] = col[i][j] = 0;
        }
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) if(mp[i][j] == 1
&& !col[i][j])
            now++, dfs(i, j);
        vector<int> ans;
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) if(!mp[i][j]) {
            set<int> s;
            int tot = 0;
            for(int k = 0; k < 4; k++) {
                int xx = i + dx[k], yy = j + dy[k];
                if(xx < 1 || xx > n || yy < 1 || yy > n) continue;
                if(mp[xx][yy] == 0) continue;
                s.insert(col[xx][yy]);
            }
            for(auto x: s) tot += cnt[x];
            ans.push_back(tot);
        }
        sort(ans.begin(), ans.end());
        printf("%d\n", ans.size());
        for(auto x: ans) printf("%d ", x);
        puts("");
    }
}

```

H XIAO7 和逗猫棒

本题提交显示 Other Error 代表 Wrong Answer 。

题目描述

猫娘「雪风」想要穿裙子！

XIAO7 告诉她：「如果你能答对以下若干个问题，我就给你买裙子。」

请你帮帮可爱的「雪风」！

具体来说，XIAO7 有 n 根逗猫棒，逗猫棒有两个属性：颜色和长度。XIAO7 想知道，能否用三根颜色互不相同的逗猫棒组成一个**非退化三角形**。

设三角形的三边长度为 $a \leq b \leq c$ ，该三角形是非退化三角形当且仅当 $a + b > c$ 。

输入格式

第一行一个正整数 n ($1 \leq n \leq 2 \times 10^5$)，表示逗猫棒的数量。

接下来 n 行，第 i ($1 \leq i \leq n$) 行两个正整数 c_i, l_i ($1 \leq c_i, l_i \leq 10^9$)，表示编号为 i 的逗猫棒的颜色和长度。

输出格式

若存在三根颜色互不相同的逗猫棒可以组成一个非退化三角形，输出一行三个互不相同的正整数 i, j, k ($1 \leq i, j, k \leq n$)，表示三根逗猫棒的编号。如果有多种可能的答案，你可以输出任意一种。

否则，输出 I would like to see xf wear a skirt every day !。

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 200005;
const int M = 55;
const int inf = 0x3f3f3f3f;
int getint() {
    char c;
    bool flag=0;
    int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0'&& c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}
const int cid[3][2] = {{0, 1}, {0, 2}, {1, 2}};
int color[N], len[N], id[N], mx[3];
bool cmp(int i, int j) {
    return len[i] < len[j];
}
void add(int x) {
    for(int i = 0; i < 3; i++)
        if(color[mx[i]] == color[x]) {
            len[mx[i]] < len[x] ? mx[i] = x : 1;
            return ;
        }
    sort(mx, mx + 3, cmp);
    len[mx[0]] < len[x] ? mx[0] = x : 1;
    return ;
}
bool check(int i, int j, int k) {
```

```

        return color[i] != color[k] && color[j] != color[k] && len[i] + len[j] >
len[k];
    }
    int main() {
        int n = getint();
        if(n < 3) {
            puts("I would like to see xf wear a skirt every day !");
            return 0;
        }
        for(int i = 1; i <= n; i++)
            scanf("%d%d", &color[i], &len[i]), id[i] = i;
        sort(id + 1, id + n + 1, cmp);
        add(id[1]), add(id[2]);
        for(int i = 3; i <= n; i++) {
            for(int j = 0; j < 3; j++)
                if(check(mx[cid[j][0]], mx[cid[j][1]], id[i])) {
                    printf("%d %d %d\n", mx[cid[j][0]], mx[cid[j][1]], id[i]);
                    return 0;
                }
            add(id[i]);
        }
        puts("I would like to see xf wear a skirt every day !");
    }
}

```

I Bellalabella 和生日礼物

本题提交显示 Other Error 代表 Wrong Answer。

题目描述

Bellalabella 最近又要过生日了，她收到了来自 n 位同学的礼物。这些礼物大多都是相同类型的礼物，第 i 个同学送的礼物为第 a_i 种礼物。Bellalabella 想知道第 $l, l+1, \dots, r$ 位同学送的礼物中有哪种礼物出现不少于 $\left\lceil \frac{r-l+1}{k} \right\rceil$ 次，其中 $\lceil x \rceil$ 表示不小于 x 的最小整数。

Bellalabella 会进行 q 次上述询问。

如果有多种礼物满足条件，Bellalabella 只想知道其中一个；如果没有满足条件的礼物，你也需要告知 Bellalabella 这一点。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^6$)，表示收到的礼物个数。

第二行 n 个正整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$)，含义同题目描述。

第三行一个正整数 q ($1 \leq q \leq 10^4$)，表示询问次数。

接下来 q 行，第 i ($1 \leq i \leq q$) 行包含三个整数 l_i, r_i, k_i ($1 \leq l_i \leq r_i \leq n, 1 \leq k_i \leq 4$)，表示第 i 个询问的参数。

输出格式

输出 q 行，第 i ($1 \leq i \leq q$) 行一个整数：

- 若存在满足条件的礼物种类，输出该礼物种类。如果有多种可能的答案，你可以输出任意一种。
- 否则，输出 -1 。

```

#include<bits/stdc++.h>
#define maxn 1000005
using namespace std;

inline int read() {
    int x = 0, f = 1;
    char ch = getchar();
    while(ch > '9' || ch < '0') {
        if(ch == '-') f = -1;
        ch = getchar();
    }
}

```

```

    }
    while(ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

vector<int> app[maxn];
int a[maxn], n, q;

int main() {
    srand(time(0));
    n = read();
    for(int i = 1; i <= n; i++) a[i] = read(), app[a[i]].push_back(i);
    q = read();
    for(int i = 1; i <= q; i++) {
        int l = read(), r = read(), k = read(), ok = (r - l + 1 + k - 1) / k;
        int ans = -1;
        for(int t = 1; t <= 60; t++) {
            int x = a[rand() % (r - l + 1) + l];
            if(lower_bound(app[x].begin(), app[x].end(), r + 1) -
upper_bound(app[x].begin(), app[x].end(), l - 1) >= ok)
                ans = x;
        }
        printf("%d\n", ans);
    }
}

```

J 小水獭和神秘动物

小水獭正在沉迷「神秘动物在哪里」，无心学习离散数学，请你帮它求出满足如下条件的方阵 $\{a_{i,j}\}_{n \times n}$ 数量：

- $a_{i,j} \in \{0, 1\}$ 对 $1 \leq i, j \leq n$ 成立。
- $a_{i,i} = 1$ 对 $1 \leq i \leq n$ 成立。
- $a_{i,j} = 1 \wedge a_{j,k} = 1 \Rightarrow a_{i,k} = 1$ 对 $1 \leq i, j, k \leq n$ 成立。
- 此外，有 m 个额外限制，第 i ($1 \leq i \leq m$) 个限制形如 $a_{x_i, y_i} = z_i$ 。

可以证明，在本题数据范围限制下，答案不超过 10^7 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 40$)，表示数据组数。

对于每组数据，一行两个非负整数 n, m ($1 \leq n \leq 7, 0 \leq m \leq n(n-1)$)，表示方阵的大小和额外限制数量。

接下来 m 行，第 i ($1 \leq i \leq m$) 行三个非负整数 x_i, y_i, z_i ($1 \leq x_i, y_i \leq n, x_i \neq y_i, 0 \leq z_i \leq 1$)，表示第 i 个限制的参数。

保证 $x_i = x_j \wedge y_i = y_j \Leftrightarrow i = j$ 对 $1 \leq i, j \leq n$ 成立，且 t 组数据的答案之和不超过 3×10^7 。

输出格式

对于每组数据，输出一行一个非负整数，表示满足条件的方阵数量。

```

#include <bits/stdc++.h>
#define maxn 10

using namespace std;

int t, n, m;

```

```

int ans;
vector<pair<int, int> > v;
int tag[maxn][maxn];
int in1[maxn], out1[maxn], in0[maxn], out0[maxn];

void dfs(int i) {
    if(i == v.size()) {
        ans++;
        return;
    }
    int x = v[i].first, y = v[i].second;
    if(tag[x][y] != 1 && !(out1[x] & in1[y])) {
        out0[x] ^= 1 << y;
        in0[y] ^= 1 << x;
        dfs(i + 1);
        out0[x] ^= 1 << y;
        in0[y] ^= 1 << x;
    }
    if(tag[x][y] != 0 && !(out0[x] & out1[y]) && !(in1[x] & in0[y])) {
        out1[x] ^= 1 << y;
        in1[y] ^= 1 << x;
        dfs(i + 1);
        out1[x] ^= 1 << y;
        in1[y] ^= 1 << x;
    }
}

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d%d", &n, &m);
        for(int i = 0; i < n; i++) for(int j = 0; j < n; j++) tag[i][j] = -1;
        while(m--) {
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            tag[x - 1][y - 1] = z;
        }
        v.clear();
        for(int i = 0; i < n; i++) for(int j = 0; j < n; j++) if(i ^ j)
v.push_back({i, j});
        ans = 0, dfs(0);
        printf("%d\n", ans);
    }
}

```

C3

A 进厂切钢条

互联网寒冬袭来，Zhoues 打算提前进厂，体验一下流水线的感觉是什么样的。

Zhoues 给一家钢条厂发送了一份自己的简历，希望可以寒假在这里实习。厂里负责招聘的工作人员正好最近也遇到了一个难题，需要一个会动态规划算法的同学帮帮忙。他恰好看到 Zhoues 的简历说自己熟练掌握《算法导论》上的动态规划问题，于是打算用这个难题考一考 Zhoues，问题如下：

- 给定一段长度为 n 英寸的钢条和一个价格表，该价格表表示长度为 i ($i = 1, 2, \dots, n$) 英寸的钢条的价格为 p_i 。求钢条切割方案，使得总销售价格最大，注意钢条的长度必须为整英寸。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^4$)，表示钢条的总长度。

第二行 n 个正整数 $p_1, p_2, \dots, p_n$ ($1 \leq p_i \leq 10^7$)，表示钢条的价格。

输出格式

一行一个正整数，表示最大总销售价格。

```
#include<bits/stdc++.h>
#define maxn 100005
typedef long long ll;
using namespace std;
inline ll read(){
    ll x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}
ll dp[maxn];
int main(){
    int n = read();
    for(int i = 1; i <= n; i++) dp[i] = read();
    for(int i = 2; i <= n; i++){
        for(int j = 1; j < i; j++) dp[i] = max(dp[i], dp[j] + dp[i - j]);
    }
    printf("%lld\n", dp[n]);
    return 0;
}
```

B 进厂添括号

Zhoues 成功进厂了，现在被通知如果想晋升进行高级流水线技师，需要进行一场额外的算法考核。Zhoues 希望你们可以帮助他通过该考核，因为他真的很想当高级技师！题目内容如下：

- 给定 n 个矩阵的链乘 $A_1 A_2 \cdots A_n$ ，你需要在不改变矩阵链顺序的前提下给矩阵添加 $n - 1$ 对合法的括号，使得矩阵相乘运算次数最少。

你只需要输出最少运算次数。

输入格式

第一行一个整数 n ($1 \leq n \leq 300$)，表示矩阵的个数。

第二行 $n + 1$ 个正整数 $a_1, a_2, \dots, a_{n+1}$ ($1 \leq a_i \leq 10^3$)，表示矩阵 A_i 的行数和列数为 a_i 和 a_{i+1} 。

输出格式

一行一个正整数，表示最少运算次数。

```
#include<bits/stdc++.h>
#define maxn 310
#define INF LLONG_MAX
typedef long long LL;
using namespace std;
LL m[maxn][maxn],s[maxn][maxn];
LL a[maxn],n;
```

```

void Matrixchain() {
    for(LL l=2; l<=n; l++)
        for(LL i=1; i<=n-l+1; i++) {
            LL j = i+l-1;
            m[i][j] = INF;
            for(LL k = i; k<j; k++) {
                LL tmp = m[i][k] + m[k+1][j] + a[i-1] * a[k] * a[j];
                if(tmp<m[i][j]) m[i][j] = tmp,s[i][j] = k;
            }
        }
}

int main() {
    scanf("%lld",&n);
    for(LL i=0; i<=n; i++)
        scanf("%lld",a+i);
    for(LL i=0; i<=n; i++)
        m[i][i] = 0;
    Matrixchain();
    printf("%lld\n", m[1][n]);
}

```

C 进厂体验流水线

Zhoues 现在是高级技师哇！招聘人员让 Zhoues 明天立马过来报道并体验厂里车间的流水线装配环节。

流水线车间的负责人也想考一下 Zhoues，具体来说，此车间有 2 条流水线，每条流水线有 m 个装配站，编号都为 $1, 2, \dots, m$ ，每条流水线同样编号的装配站都执行相同的功能。拼装一个成品要按顺序经过 m 个装配站才能加工完成，经过第 i 条流水线的第 j 个装配站要花费 $p_{i,j}$ 的时间，从第 i 条流水线移动到第 j 条流水线要花费 $t_{i,j}$ 的时间，请问制造一个成品最少时间是多少，而且具体在流水线装配的过程是什么呢？

输入格式

第一行一个正整数 m ($2 \leq m \leq 100$)，表示每条流水线有 m 个装配站。

接下来两行，每行 m 个正整数 $p_{i,1}, p_{i,2}, \dots, p_{i,m}$ ($0 \leq p_{i,j} \leq 100$)，表示经过第 i 条流水线的第 j 个装配站要花费的时间。

接下来两行，每行两个正整数 $t_{i,1}, t_{i,2}$ ($0 \leq t_{i,j} \leq 10$)，表示从第 i 条流水线移动到第 j 条流水线所花费的时间。

输出格式

第一行一个非负整数，表示制造一个成品的最少时间。

接下来 m 行，表示一个成品在流水线装配的具体过程，要求按装配站编号从小到大的顺序输出，输出样式形如 `Station%d, Line%d`，数据保证方案唯一。

```

#include <iostream>
#include <cstring>
#include <climits>
#include <vector>
using namespace std;
#define maxn 100005
typedef long long ll;
int m;
ll dp[3][maxn];
int line[3][maxn];
int work[3][maxn];
int to[3][3];
int l;
vector<pair<int, int>> v;

inline void init()
{
    for (int i = 1; i <= 2; ++i)
    {

```

```

        for (int j = 1; j <= m; ++j)
        {
            dp[i][j] = LLONG_MAX;
        }
    }
}

inline ll buildDP()
{
    for (int i = 1; i <= 2; ++i)
    {
        dp[i][1] = work[i][1];
    }

    // 每一列
    for (int j = 2; j <= m; ++j)
    {
        // 每一条流水线行数
        for (int i = 1; i <= 2; ++i)
        {
            // 前一条流水线行数
            for (int k = 1; k <= 2; ++k)
            {
                if (dp[i][j] > work[i][j] + dp[k][j - 1] + to[k][i])
                {
                    dp[i][j] = work[i][j] + dp[k][j - 1] + to[k][i];
                    line[i][j] = k;
                }
            }
        }
    }

    ll result = LLONG_MAX;
    for (int i = 1; i <= 2; ++i)
    {
        result = min(dp[i][m], result);
    }

    if (result == dp[1][m])
    {
        l = 1;
    }
    else
    {
        l = 2;
    }
    return result;
}

void searchDP()
{
    v.push_back(make_pair(0, 0));
    for (int j = m; j >= 1; j--)
    {
        v.push_back(make_pair(j, 1));
        l = line[l][j];
    }
    for (int j = m; j >= 1; j--)
    {

```

```

        printf("Station%d, Line%d\n", v[j].first, v[j].second);
    }
}

int main()
{
    scanf("%d", &m);
    init();
    for (int i = 1; i <= 2; ++i)
    {
        for (int j = 1; j <= m; ++j)
        {
            scanf("%d", &work[i][j]);
        }
    }
    for (int i = 1; i <= 2; ++i)
    {
        for (int j = 1; j <= 2; ++j)
        {
            scanf("%d", &to[i][j]);
        }
    }
    printf("%lld\n", buildDP());
    searchDP();
}

```

D 小水獭和最大值 I

小水獭正在学习动态规划。

对于给定的整数序列 $a_1, a_2, \dots, a_n$ ，小水獭想知道不相邻子序列元素和的最大值是多少。形式化地说，对于所有非空整数序列 $x_1, x_2, \dots, x_m$ ，满足 $1 \leq x_i + 1 < x_{i+1} \leq n$ 对 $1 \leq i < m$ 成立，求解 $\sum_{i=1}^m a_{x_i}$ 的最大值。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^6$)，表示序列的长度。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$)，表示序列的元素。

输出格式

对于每组数据，输出一行一个整数，表示不相邻子序列元素和的最大值。

```

#include <bits/stdc++.h>
#define maxn 1000086

using namespace std;

typedef long long ll;

int t, n;
int a[maxn];
ll f[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);

```

```

    for(int i = 1;i <= n;i++) scanf("%d", &a[i]);
    f[1] = a[1];
    if(n >= 2) f[2] = max(a[1], a[2]);
    for(int i = 3;i <= n;i++){
        f[i] = max({f[i - 1], f[i - 2] + a[i], 1ll * a[i]});
    }
    printf("%lld\n", f[n]);
}
}

```

E 小水獭和最大值 II

小水獭正在学习动态规划。

给定一个 $n \times m$ 的矩阵 $\{a_{i,j}\}_{n \times m}$ ，小水獭初始位于 $(1, 1)$ ，目标位置为 (n, m) ，它每次可以向下移动或向右移动，即从 (x, y) 移动到 $(x + 1, y)$ 或 $(x, y + 1)$ ，但不能移动到矩阵之外，它想知道经过位置元素之和的最大值是多少。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($2 \leq n, m \leq 10^3$)，含义同题目描述。

接下来 n 行，第 i ($1 \leq i \leq n$) 行 m 个整数 $a_{i,1}, a_{i,2}, \dots, a_{i,m}$ ($-10^9 \leq a_{i,j} \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个整数，表示经过位置元素之和的最大值。

```

#include <bits/stdc++.h>
#define maxn 1086

using namespace std;

typedef long long ll;

int t, n, m;
int a[maxn][maxn];
ll f[maxn][maxn];

int main(){
    memset(f, -0x3f, sizeof(f)), f[0][1] = 0;
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1;i <= n;i++) for(int j = 1;j <= m;j++) scanf("%d", &a[i]
[j]);
        for(int i = 1;i <= n;i++) for(int j = 1;j <= m;j++){
            f[i][j] = max(f[i - 1][j], f[i][j - 1]) + a[i][j];
        }
        printf("%lld\n", f[n][m]);
    }
}

```

F XIAO7 和数集（简单版）

由于所有的喵币都分给「雪风」和「星瑶」了，猫娘「凉宫秋小七」对此很不高兴又哭又闹，XIAO7 只好告诉她，若你能答对以下问题，我就把上一题扔掉的喵币捡回来偷偷给你。

具体来说，对于不含前导零的正整数 x ，定义 $S(x)$ 表示 x 的十进制表示下的每一位数字组成的集合，例如：

- $S(11523) = \{1, 2, 3, 5\}$
- $S(67654) = \{4, 5, 6, 7\}$

对于给定的 n ，XIAO7 想知道有多少对正整数 (x, y) 满足 $1 \leq x < y \leq n$ 且 $S(x) = S(y)$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，一行一个正整数 n ($1 \leq n \leq 10^6$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示答案。

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<assert.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 1000005;
const int inf = 0x3f3f3f3f;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
int num[N];
int S(int x){
    int state = 0;
    for(; x; x /= 10)
        state |= 1 << (x % 10);
    return state;
}
int main(){

    int q = getint();

    while(q --) {
        int n = getint();
        assert(1 <= n && n <= 1000000);

        ll ans = 0;
        memset(num, 0, sizeof num);
        for(int i = 1; i <= n; i++){
            int s = S(i);
            ans += num[s], num[s] ++;
        }
        cout << ans << endl;
    }
```

```
}  
}
```

G 7sozx 特有的字符串

7sozx 特别喜欢字符串 $T = 1145141919810$ 。

对于给定的字符串 S ，7sozx 想让你帮它求出一个 S 的划分 $s_1, s_2, \dots, s_m$ ，满足：

- 将 $s_1, s_2, \dots, s_m$ 顺序拼接得到的字符串和 S 相同。
- $\sum_{i=1}^m f(s_i, T)$ 最大。

其中 $f(a, b)$ 表示字符串 a 和 b 相同的位置数量。形式化地说， $f(a, b)$ 是满足 $a_i = b_i$ 的 i 的数量，其中 $i = 1, 2, \dots, \min(|a|, |b|)$ 。例如：

- 当 $a = \underline{1234567}, b = \underline{1534967}$ 时， $f(a, b) = 5$ ，相同的位置用下划线标出。
- 当 $a = \underline{1234567}, b = \underline{153496789}$ 时， $f(a, b) = 5$ ，相同的位置用下划线标出。

你只需要输出 $\sum_{i=1}^m f(s_i, T)$ 的最大值。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行一个十进制数字串 s ($1 \leq |s| \leq 10^5$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示答案。

```
#include <bits/stdc++.h>  
#define maxn 100086  
  
using namespace std;  
  
int t, n;  
char s[maxn];  
char T[14] = "1145141919810";  
int f[maxn][14];  
  
int main(){  
    scanf("%d", &t);  
    while(t--){  
        scanf("%s", s + 1);  
        n = strlen(s + 1);  
        memset(f, -0x3f, sizeof(f));  
        f[0][0] = 0;  
        for(int i = 1; i <= n; i++){  
            for(int j = 0; j < 14; j++){  
                if(j < 13) f[i][j + 1] = max(f[i][j + 1], f[i - 1][j] + (s[i] ==  
T[j]));  
                f[i][1] = max(f[i][1], f[i - 1][j] + (s[i] == T[0]));  
            }  
        }  
        int ans = 0;  
        for(int i = 0; i < 14; i++) ans = max(ans, f[n][i]);  
        printf("%d\n", ans);  
    }  
}
```

H XIAO7 和喵币

XIAO7 有 n 个喵币，面值分别为 $a_1, a_2, \dots, a_n$ 。XIAO7 要将它们分给「雪风」和「星瑶」两只猫娘，但如果两只猫娘分到的喵币总面值不同，她们就会打架，因此多余的不能均分的喵币只能被扔掉，XIAO7 想知道需要扔掉喵币的面值之和最少是多少。

注意，每张喵币的面值是固定的，不能进行拆分。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^3$)，表示喵币的数量。

第二行 n 个非负整数 $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^4$)，表示喵币的面值。

保证 $\sum_{i=1}^n a_i \leq 10^4$ 。

输出格式

第一行一个非负整数，表示最少需要扔掉的喵币面值总和。

第二行若干个正整数，表示需要扔掉的喵币的序号。如果有多种可能的答案，你可以输出任意一种。

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<assert.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 1005;
const int M = 10005;
const int inf = 0x3f3f3f3f;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
int a[N], sum[N], f[N][M];
vector<int> last;
void dfs(int n, int d){
    if(n < 1) return ;
    else if(f[n][d] == f[n - 1][d] + a[n]){
        last.push_back(n);
        dfs(n - 1, d);
    }
    else if(f[n][d] == f[n - 1][d + a[n]])
        dfs(n - 1, d + a[n]);
    else dfs(n - 1, abs(d - a[n]));
}
int main(){
    int n = getint();
    assert(n <= 1000);
    for(int i = 1; i <= n; i ++){
        sum[i] = sum[i - 1] + (a[i] = getint());
        assert(sum[n] <= 10000);
    }
}
```

```

memset(f, 0x3f, sizeof(f));
f[0][0] = 0;
for(int i = 1; i <= n; i ++){
    for(int j = 0; j <= sum[i - 1]; j ++){
        f[i][j] = min(f[i][j], f[i - 1][j] + a[i]);
        f[i][j + a[i]] = min(f[i][j + a[i]], f[i - 1][j]);
        f[i][abs(j - a[i])] = min(f[i][abs(j - a[i])], f[i - 1][j]);
    }

    cout << f[n][0] << endl;

    dfs(n, 0);
    for(auto v : last)
        printf("%d ", v);
}

```

I 任务达人莫卡 II

莫卡穿越到了一个新的世界并且遇到了一位名为任务达人的 NPC。

这个新的世界一共有 n 个城市，编号从 1 到 n ，莫卡起初位于编号为 k 的城市。第 i 天任务达人会给莫卡派送一个任务，该任务需要前往编号为 c_i 的城市完成，莫卡可以选择接受或者放弃这个任务，如果莫卡接受了任务，若莫卡当前即位于编号为 c_i 的城市，则可以获得 a_{c_i} 的收益，否则莫卡会前往编号为 c_i 的城市（在城市间移动是没有花费的），并获得 b_{c_i} 的收益，完成任务后莫卡会停留在编号为 c_i 的城市。如果莫卡放弃了任务，她将不会获得任何收益并会停留在当前所在城市。

莫卡想请你帮她计算出 m 天后自己可以获得的最大收益。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，代表数据组数。

对于每组数据，第一行两个正整数 n, k ($3 \leq n \leq 10^5$)，表示城市个数和起始城市编号。

第二行 n 个正整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$)，含义同题目描述。

第三行 n 个正整数 $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^9$)，含义同题目描述。

第四行一个正整数 m ($1 \leq m \leq 10^5$)，表示执行任务的天数。

接下来 m 行，每行一个正整数 c_i ($1 \leq c_i \leq n$)，表示第 i 天任务的城市编号。

输出格式

对于每组数据，输出一行一个正整数，表示 m 天后能够获得的最大收益。

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long

multiset<ll> s;
const int N = 1e5+10;
ll a[N];
ll b[N];
ll dp[N];
int main(){
    int t;
    scanf("%d",&t);
    while(t--){
        s.clear();
        int n, m, k;
        scanf("%d %d", &n, &k);
        for (int i = 1; i <= n; i++) {
            scanf("%lld", a + i);

```

```

        dp[i]=-2e9;
    }
    for (int i = 1; i <= n; i++) {
        scanf("%lld", b + i);
    }
    dp[k]=0;
    scanf("%d", &m);
    for (int i = 1; i <= n; i++) {
        s.insert(dp[i]);
    }
    for (int i = 1; i <= m; i++) {
        int c;
        scanf("%d", &c);
        auto itt = s.find(dp[c]);
        s.erase(itt);
        auto it = s.end();
        it--;
        dp[c] = max(dp[c] + a[c], (*it) + b[c]);
        s.insert(dp[c]);
    }
    auto it = s.end();
    it--;
    printf("%lld\n", *it);
}
}

```

J 小水獭和电影院（简单版）

小水獭正在学习动态规划，但想去看个电影放松一下。

电影院可以被抽象为一个 $n \times m$ 的矩阵 $\{a_{i,j}\}_{n \times m}$ ，如果电影院第 i 行第 j 列的座位有人，则 $a_{i,j} = 1$ ，否则 $a_{i,j} = 0$ 。

众所周知，为防控疫情，相邻座位不能同时有人。形式化地说， $(a_{i,j} = 1 \wedge a_{i',j'} = 1) \Rightarrow (|i - i'| + |j - j'| \neq 1)$ 。

小水獭想知道，有多少种不同的矩阵所对应的电影院是满足疫情防控要求的，由于答案可能很大，你只需要输出答案对 2^{32} 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行两个正整数 n, m ($1 \leq nm \leq 400$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示答案对 2^{32} 取模后的结果。

```

#include <bits/stdc++.h>
#define maxn 20

using namespace std;

int t, n, m;
unsigned f[1 << maxn], g[1 << maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        if(n < m) swap(n, m);
        for(int i = 0; i < (1 << m); i++) f[i] = g[i] = 0;
    }
}

```

```

    f[0] = 1;
    for(int i = 0; i < n; i++) for(int j = 0; j < m; j++){
        for(int s = 0; s < (1 << m); s++){
            int ss = (s | (1 << j)) ^ (1 << j);
            g[ss] += f[s];
            if((j == 0 || !(s & (1 << (j - 1)))) && !(s & (1 << j))) g[ss |
(1 << j)] += f[s];
        }
        for(int s = 0; s < (1 << m); s++) f[s] = g[s], g[s] = 0;
    }
    unsigned ans = 0;
    for(int i = 0; i < (1 << m); i++) ans += f[i];
    printf("%u\n", ans);
}
}

```